

Fondamenti di Informatica T2

Lab07 – Calendario appuntamenti

Corso di Laurea in Ingegneria Informatica

Anno accademico 2023/2024

Prof. ROBERTA CALEGARI

Prof. AMBRA MOLESINI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

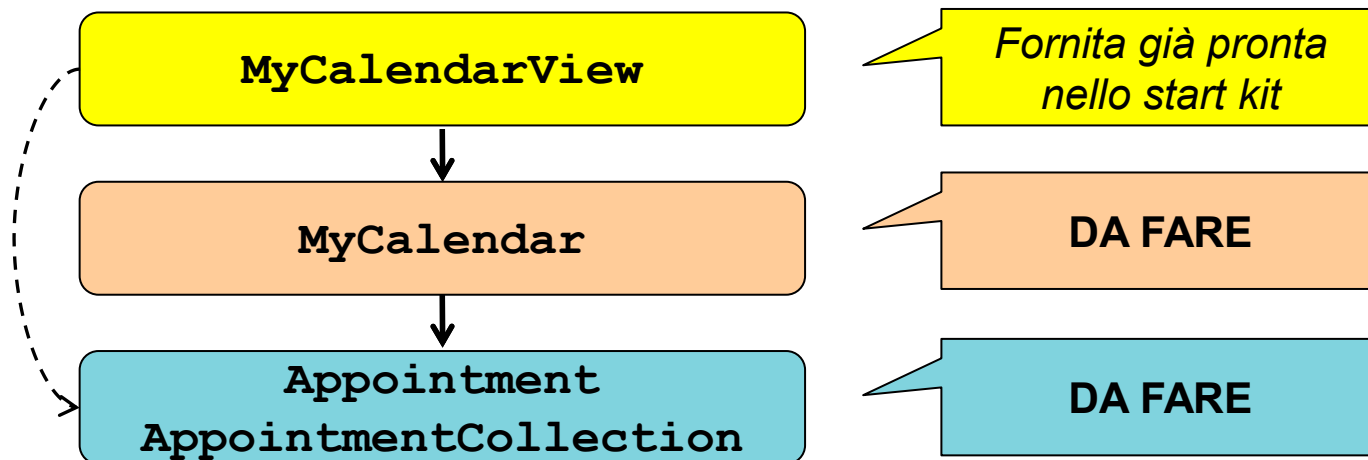


Calendario Appuntamenti

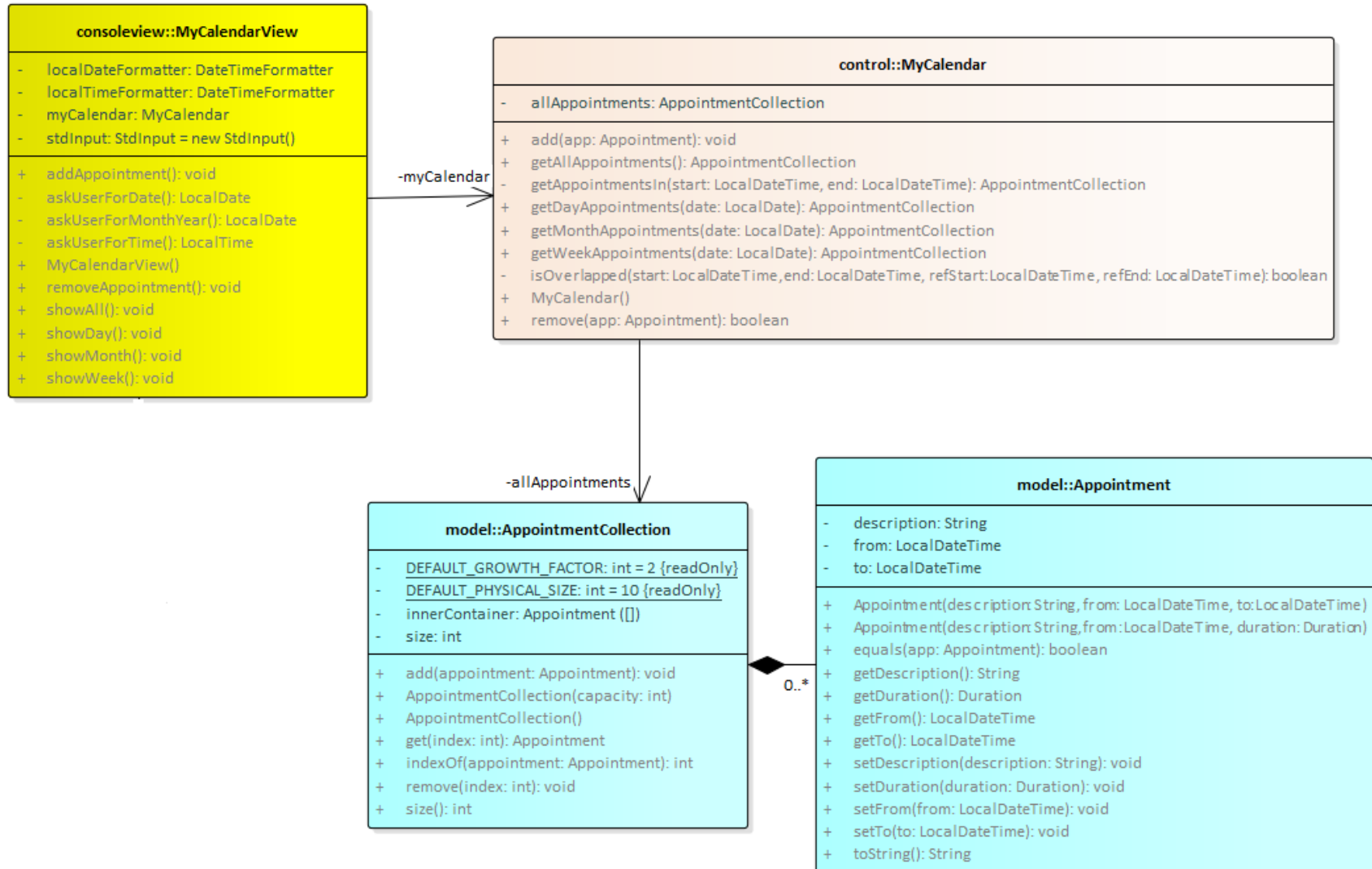
- **OBIETTIVO: realizzare una applicazione per la gestione di un calendario di appuntamenti**
- L'applicazione dovrà offrire le seguenti funzionalità:
 - **Inserire** un nuovo appuntamento
 - **Cancellare** un appuntamento
 - **Mostrare** tutti gli appuntamenti
 - **Ottenere** la lista degli appuntamenti:
 - Giornaliera
 - Mensile
- **Ma... cos'è esattamente un *APPUNTAMENTO* ?**

Calendario appuntamenti con MVC

- Nel caso della nostra applicazione Calendario appuntamenti:
 - **Model:** due classi, **Appointment** e **AppointmentCollection**
 - **View:** classe **MyCalendarView** (*fornita già pronta nello start kit*)
 - **Controller:** classe **MyCalendar**



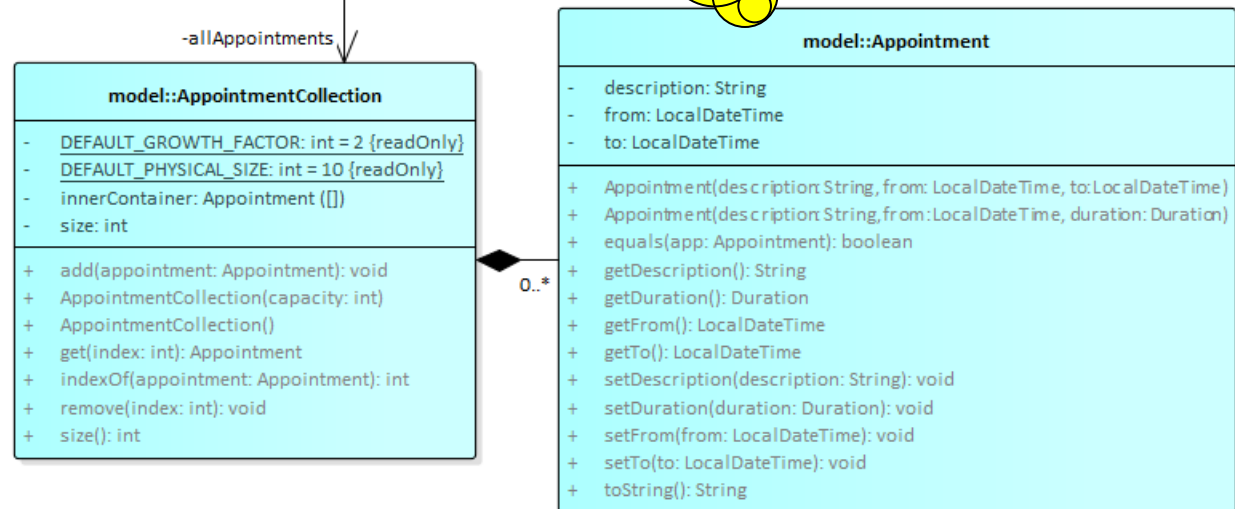
Organizzazione dell'applicazione



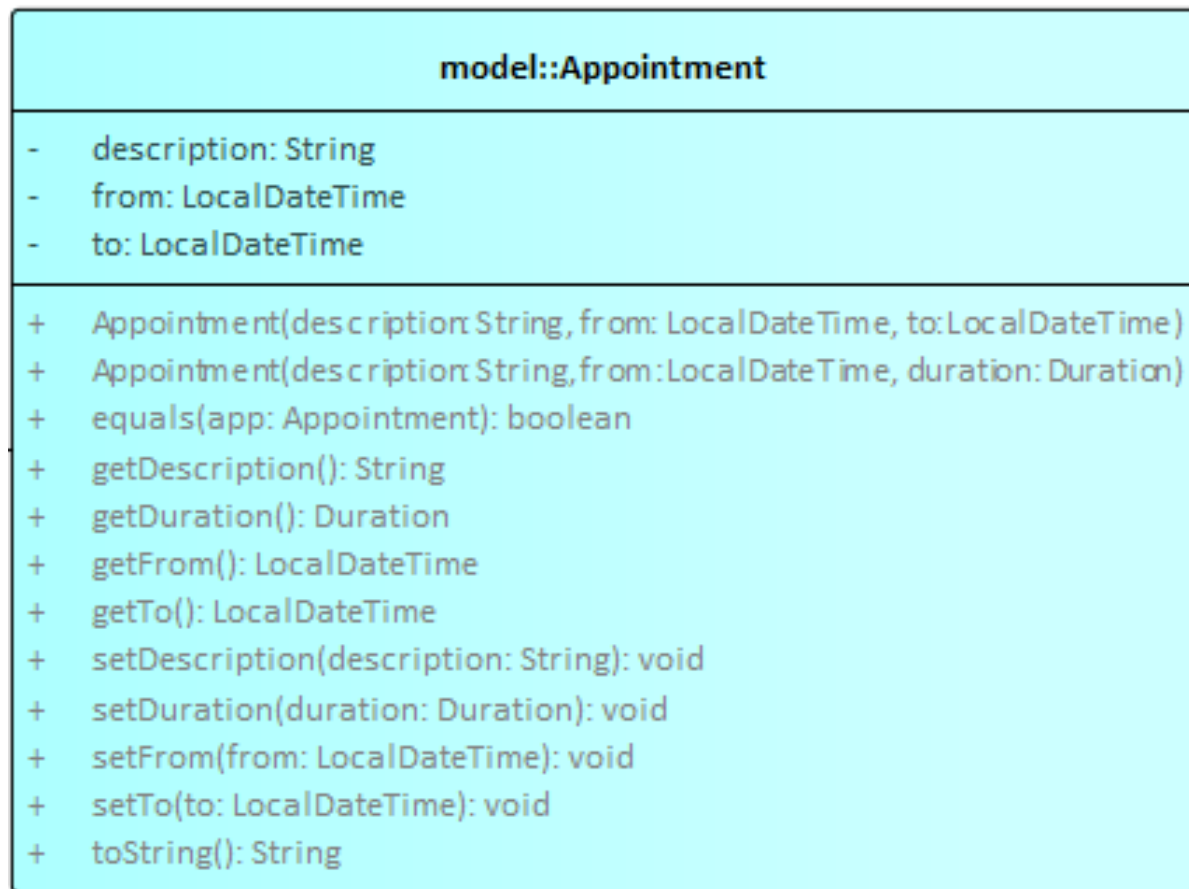
Organizzazione dell'applicazione

Suggerimento: in presenza di diverse classi da realizzare **si parte sempre da quella con meno legami** (dipendenze) con le altre classi, in questo caso si parte da **Appointment**

```
consoleview:  
- LocalDateForm  
- LocalTimeForm  
- ...  
- ask...  
- ...  
+ ...  
+ ...  
+ showDay(): void  
+ showMonth(): void  
+ showWeek(): void
```



Appointment UML



NON INVENTARE SPECIFICHE! Seguire il progetto stabilito!

AppointmentCollection

model::AppointmentCollection

- DEFAULT_GROWTH_FACTOR: int = 2 {readOnly}
- DEFAULT_PHYSICAL_SIZE: int = 10 {readOnly}
- innerContainer: Appointment ([])
- size: int

- + add(appointment: Appointment): void
- + AppointmentCollection(capacity: int)
- + AppointmentCollection()
- + get(index: int): Appointment
- + indexOf(appointment: Appointment): int
- + remove(index: int): void
- + size(): int

NON INVENTARE SPECIFICHE! Seguire il progetto stabilito!

MyCalendar UML

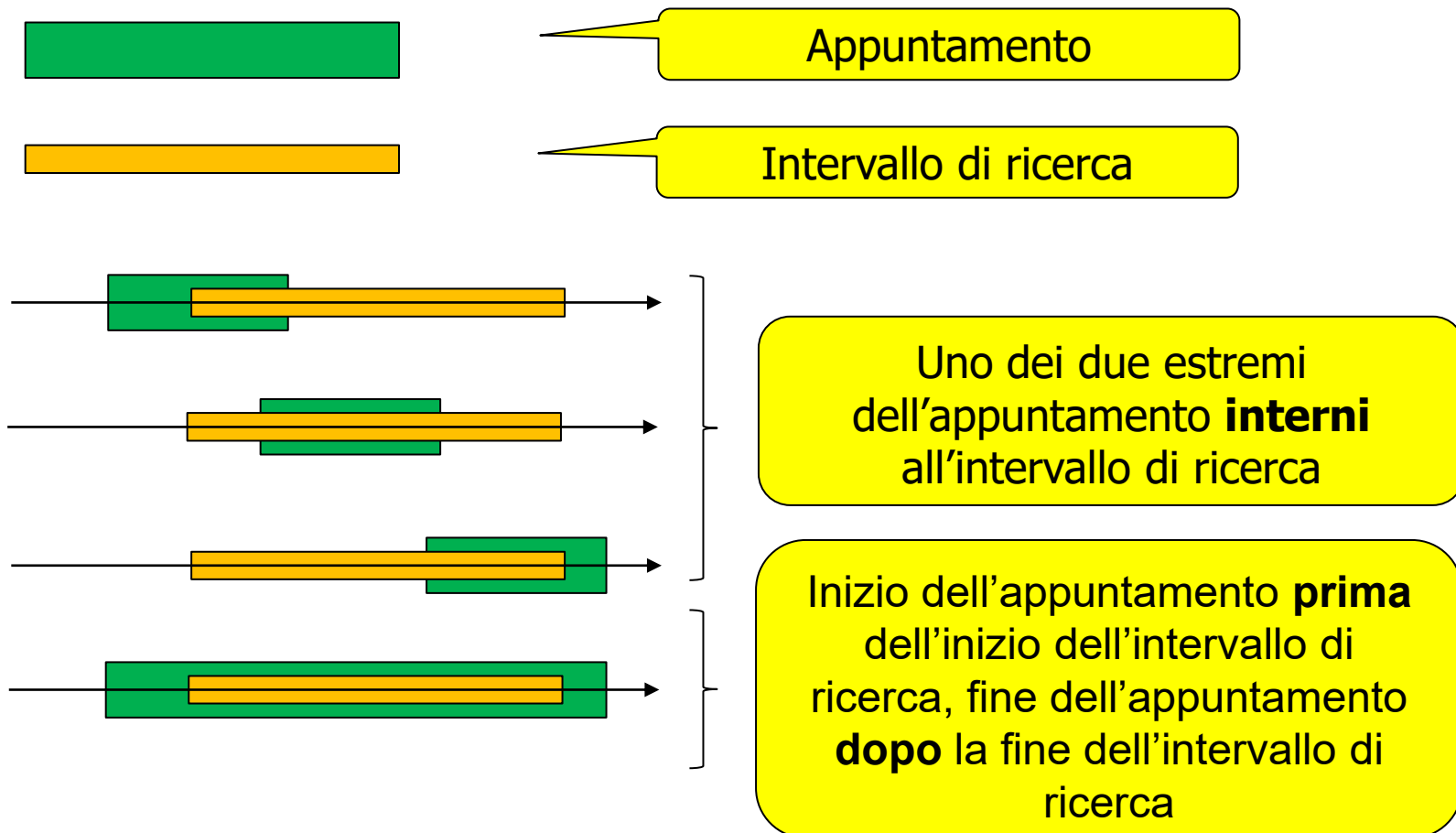
control::MyCalendar

```
- allAppointments: AppointmentCollection  
  
+ add(app: Appointment): void  
+ getAllAppointments(): AppointmentCollection  
- getAppointmentsIn(start: LocalDateTime, end: LocalDateTime): AppointmentCollection  
+ getDayAppointments(date: LocalDate): AppointmentCollection  
+ getMonthAppointments(date: LocalDate): AppointmentCollection  
+ getWeekAppointments(date: LocalDate): AppointmentCollection  
- isOverlapped(start: LocalDateTime, end: LocalDateTime, refStart: LocalDateTime, refEnd: LocalDateTime): boolean  
+ MyCalendar()  
+ remove(app: Appointment): boolean
```

NON INVENTARE SPECIFICHE! Seguire il progetto stabilito!

Algoritmica

- Sovrapposizione di intervalli: quattro casi possibili

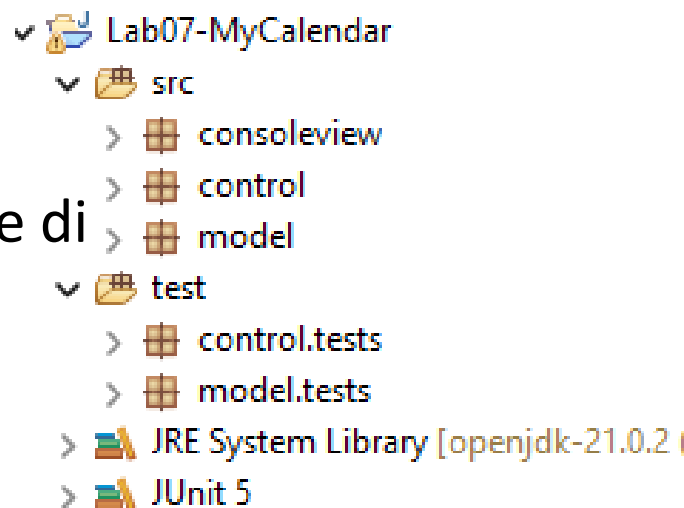


Package & Start kit

- Strutturiamo il nostro progetto seguendo la separazione logica suggerita dal Pattern MVC

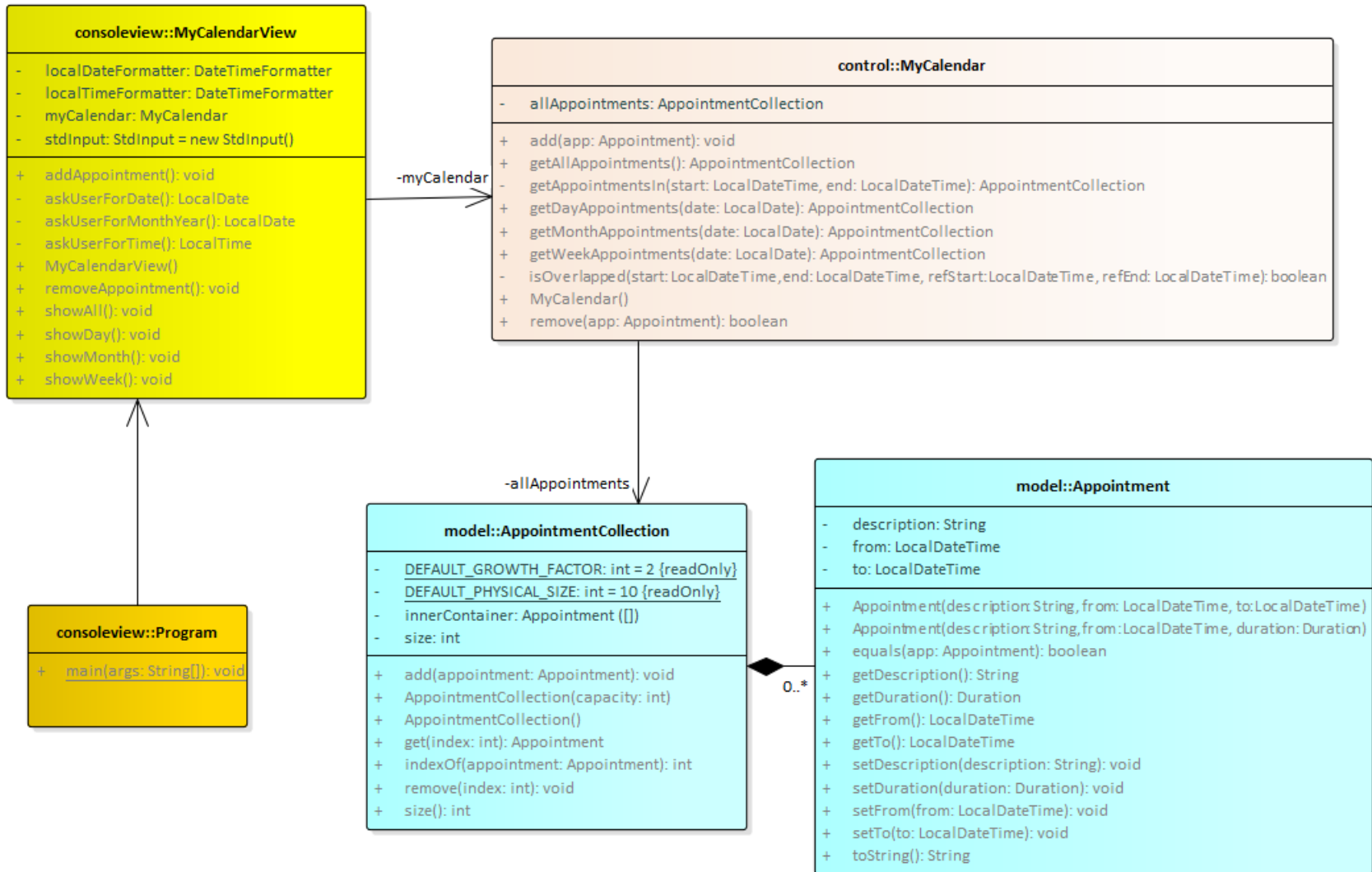
- **Due diversi source folder** per separare i sorgenti e le classi di test
→ una buona organizzazione dell'ambiente di lavoro aiuta anche chi verrà dopo di noi ad «orientarsi» meglio nel nostro lavoro

- Il folder «test» lo trovate già pronto nello startkit con i test JUnit per testare le vostre classi

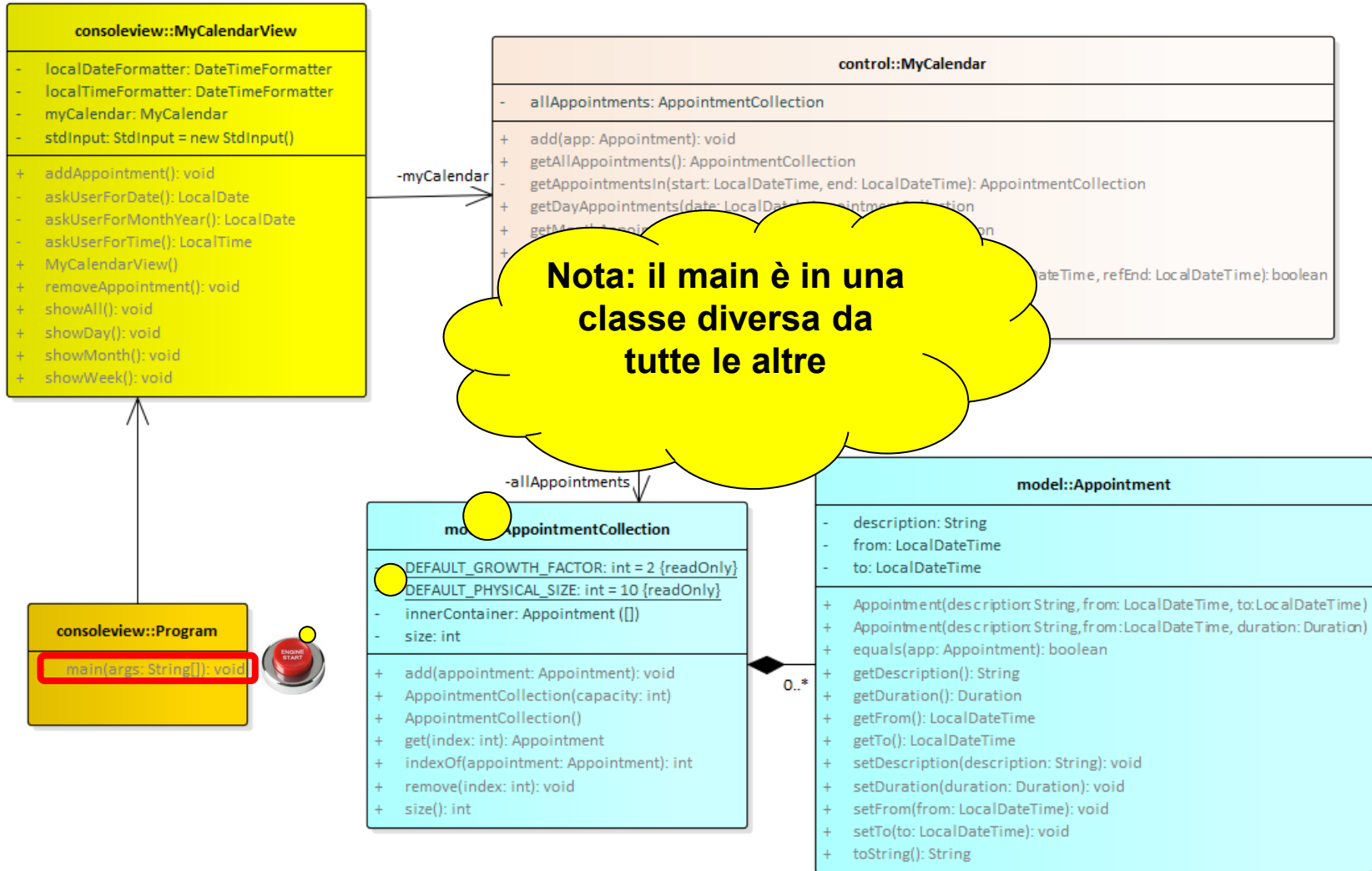


OCCHIO ai nomi dei package, delle classi e dei metodi, altrimenti i test ☹

Organizzazione dell'applicazione



Organizzazione dell'applicazione





Cosa dovete fare

- Realizzare e collaudare nell'ordine le tre classi:
 - **Appointment**
 - **AppointmentCollection**
 - **MyCalendar**
- Eventuale funzionalità opzionale:
 - ottenere la lista degli appuntamenti settimanali

Buon Lavoro!!!!